Lesson 13: Dynamic Blocks

Learning Objectives

- Understand what dynamic blocks are and when to use them
- Generate nested configuration blocks dynamically
- Use `for_each` and `iterator` inside dynamic blocks
- Combine dynamic blocks with complex variable types

---

1. What are Dynamic Blocks?

Some Terraform resources accept **nested configuration blocks** — repeating structures inside the resource body. `dynamic` blocks let you generate these from variables or other data.

```
# Without dynamic — hardcoded blocks
resource "aws_security_group" "main" {
  name = "web-sg"

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# With dynamic — generated blocks
resource "aws_security_group" "main" {
  name = "web-sg"

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.from
      to_port     = ingress.value.to
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}
```

2. Dynamic Block Syntax

```
dynamic "<block_type>" {
  for_each = <collection>  # set, map, or list

  # Optional: custom iterator name (defaults to block_type)
  iterator = "custom_name"

  content {
    # Use each.value, each.key, or custom_name.value
    attribute = <expression>
  }
}
```

Part	Description
`dynamic`	Keyword introducing a dynamic block
`"<block_type>"`	The name of the nested block (e.g., `"ingress"`, `"attribute"`, `"logging"`)
`for_each`	The collection to iterate over
`iterator`	Optional custom name for the iterator (defaults to the block type name)
`content`	The body of each generated block

Iterator Access

```
# Default iterator (same as block_type name)
dynamic "ingress" {
  for_each = var.rules
  content {
    from_port = ingress.value.from  # block_type.value
  }
}

# Custom iterator
dynamic "ingress" {
  for_each = var.rules
  iterator = "rule"
  content {
    from_port = rule.value.from  # custom_name.value
  }
}
```

3. Dynamic Blocks with Lists

```
variable "ingress_rules" {
  description = "Security group ingress rules"
  type = list(object({
    from        = number
    to          = number
    protocol    = string
    cidr_blocks = list(string)
  }))
  default = [
    { from = 80, to = 80, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] },
    { from = 443, to = 443, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"]
},
  ]
}

resource "aws_security_group" "main" {
  name        = "web-sg"
  description = "Web security group"

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.from
      to_port     = ingress.value.to
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}
```

---

4. Dynamic Blocks with Maps

```
variable "queue_configs" {
  description = "Queue configurations with tags"
  type = map(object({
    delay = number
    priority = string
  }))
  default = {
    orders  = { delay = 0, priority = "high" }
    billing = { delay = 5, priority = "critical" }
  }
}

resource "aws_sqs_queue" "main" {
  for_each = var.queue_configs
  name     = "${each.key}-queue"
}
```

Dynamic blocks are especially useful with **SNS Topic subscriptions** and **S3 bucket configurations**:

```
resource "aws_sns_topic" "main" {
  name = "alerts-topic"

  dynamic "subscription" {
    for_each = var.subscriptions
    content {
      protocol = subscription.value.protocol
      endpoint = subscription.value.endpoint
    }
  }
}
```

---

5. Dynamic Blocks with Conditionals

You can conditionally include dynamic blocks by using an empty collection:

```
variable "enable_logging" {
  type    = bool
  default = false
}

resource "aws_s3_bucket" "main" {
  bucket = "logging-bucket"

  dynamic "logging" {
    # Empty list = no blocks generated
    for_each = var.enable_logging ? [1] : []
    content {
      target_bucket = aws_s3_bucket.logs.id
      target_prefix = "log/"
    }
  }
}
```

---

6. Nested Dynamic Blocks

Dynamic blocks can be nested (though this gets complex quickly):

```
dynamic "rule" {
  for_each = var.lifecycle_rules

  content {
    id     = rule.value.id
    status = rule.value.enabled ? "Enabled" : "Disabled"
```

```
    dynamic "expiration" {
      for_each = rule.value.expiration_days != null ? [1] : []
      content {
        days = rule.value.expiration_days
      }
    }

    dynamic "transition" {
      for_each = rule.value.transitions
      content {
        days          = transition.value.days
        storage_class = transition.value.storage_class
      }
    }
  }
}
```

---

7. Limitations

Limitation	Explanation
Nested blocks only	Dynamic blocks only work for nested config blocks, not top-level arguments
Read-only inside content	You cannot reference resource attributes that depend on the dynamic block itself
Complexity	Deeply nested dynamics are hard to read — consider using modules instead
No `count` inside	Use `for_each` only; `count` is not supported in dynamic blocks

---

8. Complete Example

```
# variables.tf
variable "ingress_rules" {
  description = "Security group ingress rules"
  type = list(object({
    from_port   = number
    to_port     = number
    protocol    = string
    cidr_blocks = list(string)
  }))
  default = [
    { from_port = 80, to_port = 80, protocol = "tcp", cidr_blocks =
["0.0.0.0/0"] },
    { from_port = 443, to_port = 443, protocol = "tcp", cidr_blocks =
["0.0.0.0/0"] },
    { from_port = 22, to_port = 22, protocol = "tcp", cidr_blocks =
```

```
["10.0.0.0/8"] },
  ]
}

variable "enable_ssh" {
  description = "Whether to include SSH ingress rule"
  type        = bool
  default     = true
}

# main.tf
locals {
  # Filter rules conditionally
  rules = var.enable_ssh ? var.ingress_rules : [
    for r in var.ingress_rules : r if r.from_port != 22
  ]
}

resource "aws_security_group" "main" {
  name        = "dynamic-sg"
  description = "Security group with dynamic ingress rules"

  dynamic "ingress" {
    for_each = local.rules
    content {
      from_port   = ingress.value.from_port
      to_port     = ingress.value.to_port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}

# outputs.tf
output "sg_id" {
  value = aws_security_group.main.id
}

output "ingress_count" {
  value = length(aws_security_group.main.ingress)
}
```

---

9. Key Takeaways

- `dynamic` blocks generate nested configuration blocks from collections
- Use `for_each` inside the dynamic block — `count` is not supported
- Access values with `block_type.value.attribute` or a custom `iterator` name
- Conditional blocks work by passing an empty collection (`[]` or `{}`)
- Dynamic blocks can be nested (one dynamic inside another)
- They work for **any** nested block type: `ingress`, `subscription`, `logging`, `attribute`, `rule`

- Keep dynamic blocks simple — if you need deep nesting, consider splitting into separate resources or modules

example and exercise repo url

https://github.com/eyu-se/terraform-training-material-with-projects

Trainer - Eyuel M.